Vivado HLS 2019.1 - Matrix Multiplication

HLS is a Hardware Level Synthesis, used to run a simulated version of the hardware based on algorithms written in C/C++. In this case a Matrix Multiplication application will be used as example.

This tutorial is based in this IITD AELD Lab8_P1: HLS IP with AXI Stream Interface for Matrix Multiplication video.

Please apply the Vivado HLS 2019.1 - Customizations.

Vivado HLS 2019.1 - Create Project

Vivado HLS 2019.1 - Syntax

During project creation the following codes were added:

Hardware Algorithm

Here is described the algorithm that will be executed by the ZedBoard. This algorithms will be interpreted by Vivado HLS and then an equivalent algorithm in machine language will be generated so the FPGA can execute its function.

`matrix_mult.h`

In the this file, main function configurations will be stored mainly the matrix size and the solution algorithm used.

```
#include <stdio.h>
#include <hls_stream.h>
#include <ap_int.h>

#define SIZE 8          // matrix size
#define solution1       // active solution

// define matrix data type
#ifdef solution1
typedef float matrix;
#endif

// define axis data structure
struct axis{
        matrix data;
        ap_uint<1> last;
};
```

```
void matrix_mult_1(hls::stream<axis> &input_stream, hls::stream<axis> &output_stream);
```

matrix_mult.cpp

In this file, solution algorithms are defined.

```
#include "matrix_mult.h"

#ifdef solution1
void matrix_mult_1(hls::stream<axis> &input_stream, hls::stream<axis> &output_stream){
	#pragma HLS INTERFACE ap_ctrl_none port=return                          //
disable unwanted messages
	#pragma HLS INTERFACE axis register both port=output_stream     //
	#pragma HLS INTERFACE axis register both port=input_stream      //
	// matrix multiplication function, A * B = C

	// matrix declaration
	matrix A[SIZE][SIZE];   // input
	matrix B[SIZE][SIZE];   // input
	matrix C[SIZE][SIZE];   // output

	// local variables
	int row, col, k;
	axis data_axis;


	// stream data read
	// matrix A
	loop_stream_read_row_A: for(row = 0; row < SIZE; row++){
		loop_stream_read_col_A: for(col = 0; col < SIZE; col++){
			A[row][col] = input_stream.read().data;
		}
	}
	// matrix B
	loop_stream_read_row_B: for(row = 0; row < SIZE; row++){
		loop_stream_read_col_B: for(col = 0; col < SIZE; col++){
			B[row][col] = input_stream.read().data;
		}
	}

	// naive matrix multiplication algorithm
	loop_mult_row: for(row = 0; row < SIZE; row++){
		loop_mult_col: for(col = 0; col < SIZE; col++){
			float sum = 0;  // same type as matrix
			loop_mult_k: for(k = 0; k < SIZE; k++){
				sum += A[row][k] * B[k][col];
			}
			C[row][col] = sum;
```

```
                }
        }

        //stream data write
        loop_stream_write_row: for(row = 0; row < SIZE; row++){
                loop_stream_write_col: for(col = 0; col < SIZE; col++){
                        data_axis.data = C[row][col];

                        if((row == SIZE-1) && (col == SIZE-1))
                                data_axis.last = 1;
                        else
                                data_axis.last = 0;
                        output_stream.write(data_axis);
                }
        }
}
#endif
```

All Vivado HLS 2019.1 - Syntax options may be applied as configuration flags. It is also strongly advised to name all loops with the flags to help read data during analysis.

In this design a single Input Axis Stream and a single Output Axis Stream is used to describe the matrix multiplication. For that reason, it is necessary to read both matrices A and B from the Input Axis Stream.

Evaluation Algorithm

Here is described the algorithm used to manage the hardware execution of the matrix multiplication algorithm. It will send and receive the matrices needed for the operation and compare the result with the expected result to ensure the operation is correctly done.

`matrix_mult_tb.h`

In this file, the benchmark algorithm is defined.

```
#include "matrix_mult.h"

// benchmark is created to have a baseline to which compare the multiplication results
void matrix_mult_naive(double A[SIZE][SIZE], double B[SIZE][SIZE], double C[SIZE]
[SIZE]);
```

`matrix_mult_tb.cpp`

In this file, main routine is defined.

```
#include "matrix_mult_tb.h"
#include <stdio.h>
#include <math.h>
#include <cstdlib>


void matrix_mult_naive(double A[SIZE][SIZE], double B[SIZE][SIZE], double C[SIZE]
[SIZE]){
        int row, col, k;

        // naive matrix multiplication algorithm
        loop_mult_row: for(row = 0; row < SIZE; row++){
                loop_mult_col: for(col = 0; col < SIZE; col++){
                        double sum = 0;
                        loop_mult_k: for(k = 0; k < SIZE; k++){
                                sum += A[row][k] * B[k][col];
                        }
                        C[row][col] = sum;
                }
        }
}


int main(){
        // matrix declaration
        double A[SIZE][SIZE];   // input
        double B[SIZE][SIZE];   // input
        double C0[SIZE][SIZE];  // output naive calculation
        double C[SIZE][SIZE];   // output

        // local variables
        int row, col, k;
        axis data_axis;

        // create random A and B matrix
        loop_rand_row: for(row = 0; row < SIZE; row++){
                loop_rand_col: for(col = 0; col < SIZE; col++){
                        A[row][col] = rand()%10;
                        B[row][col] = rand()%10;
                }
        }

        // call benchmark function
        matrix_mult_naive(A, B, C0);

        //stream data write
        hls::stream<axis> input_stream, output_stream;

        loop_stream_write_A_row: for(row = 0; row < SIZE; row++){
```

```
                loop_stream_write_A_col: for(col = 0; col < SIZE; col++){
                        data_axis.data = A[row][col];

                        if((row == SIZE-1) && (col == SIZE-1))
                                data_axis.last = 1;
                        else
                                data_axis.last = 0;
                        input_stream.write(data_axis);
                }
        }
        loop_stream_write_B_row: for(row = 0; row < SIZE; row++){
                loop_stream_write_B_col: for(col = 0; col < SIZE; col++){
                        data_axis.data = B[row][col];

                        if((row == SIZE-1) && (col == SIZE-1))
                                data_axis.last = 1;
                        else
                                data_axis.last = 0;
                        input_stream.write(data_axis);
                }
        }

        // algorithm call, hardware execution
        #ifdef solution1
                matrix_mult_1(input_stream, output_stream);
        #endif


        // stream data read
        loop_stream_read_row: for(row = 0; row < SIZE; row++){
                loop_stream_read_col: for(col = 0; col < SIZE; col++){
                        C[row][col] = output_stream.read().data;
                }
        }

        // results comparison
        loop_results_row: for(row = 0; row < SIZE; row++){
                loop_results_col: for(col = 0; col < SIZE; col++){
                        if(fabs(C0[row][col] - C[row][col]) > 0.01){
                                printf("error. C0[%d][%d] != C[%d][%d], with %f !=
%f.", row, col, row, col, C0[row][col], C[row][col]);
                                return 1;
                        }
                }
        }
}
```

Vivado HLS 2019.1 - Analysis

Vivado HLS 2019.1 - Wave Viewer

Vivado HLS 2019.1 - HLS Pragma Optimizations

Raw Results

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	8.510	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
2564	2564	2564	2564	none

Detail

Instance

Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- loop_stream_read_row_A	80	80	10	-	-	8	no
+ loop_stream_read_col_A	8	8	1	-	-	8	no
- loop_stream_read_row_B	80	80	10	-	-	8	no
+ loop_stream_read_col_B	8	8	1	-	-	8	no
- loop_mult_row	2192	2192	274	-	-	8	no
+ loop_mult_col	272	272	34	-	-	8	no
++ loop_mult_k	32	32	4	-	-	8	no
- loop_stream_write_row	208	208	26	-	-	8	no
+ loop_stream_write_col	24	24	3	-	-	8	no

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	3	0	427	-
FIFO	-	-	-	-	-
Instance	-	-	-	-	-
Memory	3	-	0	0	0
Multiplexer	-	-	-	305	-
Register	-	-	379	-	-
Total	3	3	379	732	0
Available	280	220	106400	53200	0
Utilization (%)	1	1	~0	1	0

Even if this solution is "not" optimized the `#pragma HLS INTERFACE ap_ctrl_none port=return // disable unwanted messages` is used in all solutions just to make execution clearer. The trip count

confirms that the matrix size is 8.

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	7.256	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
6148	6148	6148	6148	none

Detail

Instance

Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- loop_stream_read_row_A	80	80	10	-	-	8	no
+ loop_stream_read_col_A	8	8	1	-	-	8	no
- loop_stream_read_row_B	80	80	10	-	-	8	no
+ loop_stream_read_col_B	8	8	1	-	-	8	no
- loop_mult_row	5776	5776	722	-	-	8	no
+ loop_mult_col	720	720	90	-	-	8	no
++ loop_mult_k	88	88	11	-	-	8	no
- loop_stream_write_row	208	208	26	-	-	8	no
+ loop_stream_write_col	24	24	3	-	-	8	no

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	368	-
FIFO	-	-	-	-	-
Instance	-	5	348	711	-
Memory	3	-	0	0	0
Multiplexer	-	-	-	349	-
Register	-	-	386	-	-
Total	3	5	734	1428	0
Available	280	220	106400	53200	0
Utilization (%)	1	2	~0	2	0

solution0

This results is the solution0 considering float variables as matrix. since omar recommended doing with int this solution will not be carried any further. it serves only for comparison reasons.

Target device: xc7z020-clg484-1

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	8.510	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
716	716	716	716	none

Detail

Instance

Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- loop_stream_read_row_A_loop_stream_read_col_A	64	64	1	1	1	64	yes
- loop_stream_read_row_B_loop_stream_read_col_B	64	64	1	1	1	64	yes
- loop_mult_row_loop_mult_col_loop_mult_k	515	515	5	1	1	512	yes
- loop_stream_write_row_loop_stream_write_col	65	65	3	1	1	64	yes

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	3	0	625	-
FIFO	-	-	-	-	-
Instance	-	-	-	-	-
Memory	3	-	0	0	0
Multiplexer	-	-	-	404	-
Register	0	-	652	128	-
Total	3	3	652	1157	0
Available	280	220	106400	53200	0
Utilization (%)	1	1	~0	2	0

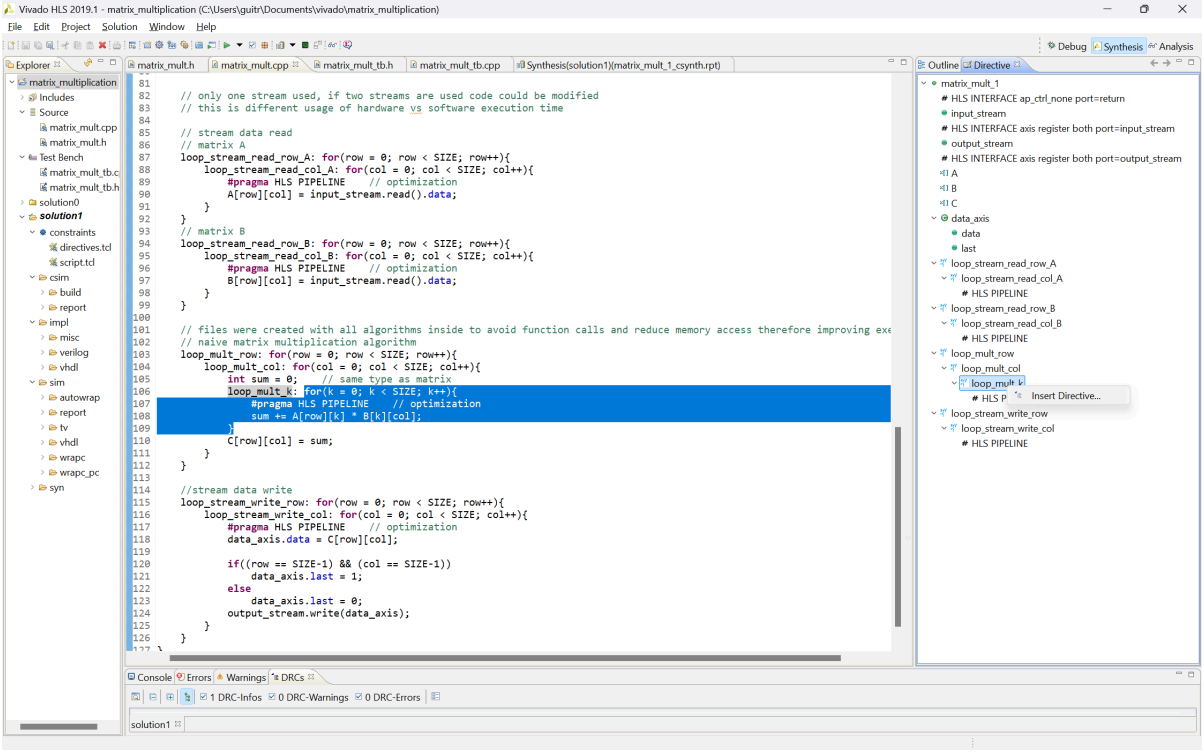
Detail

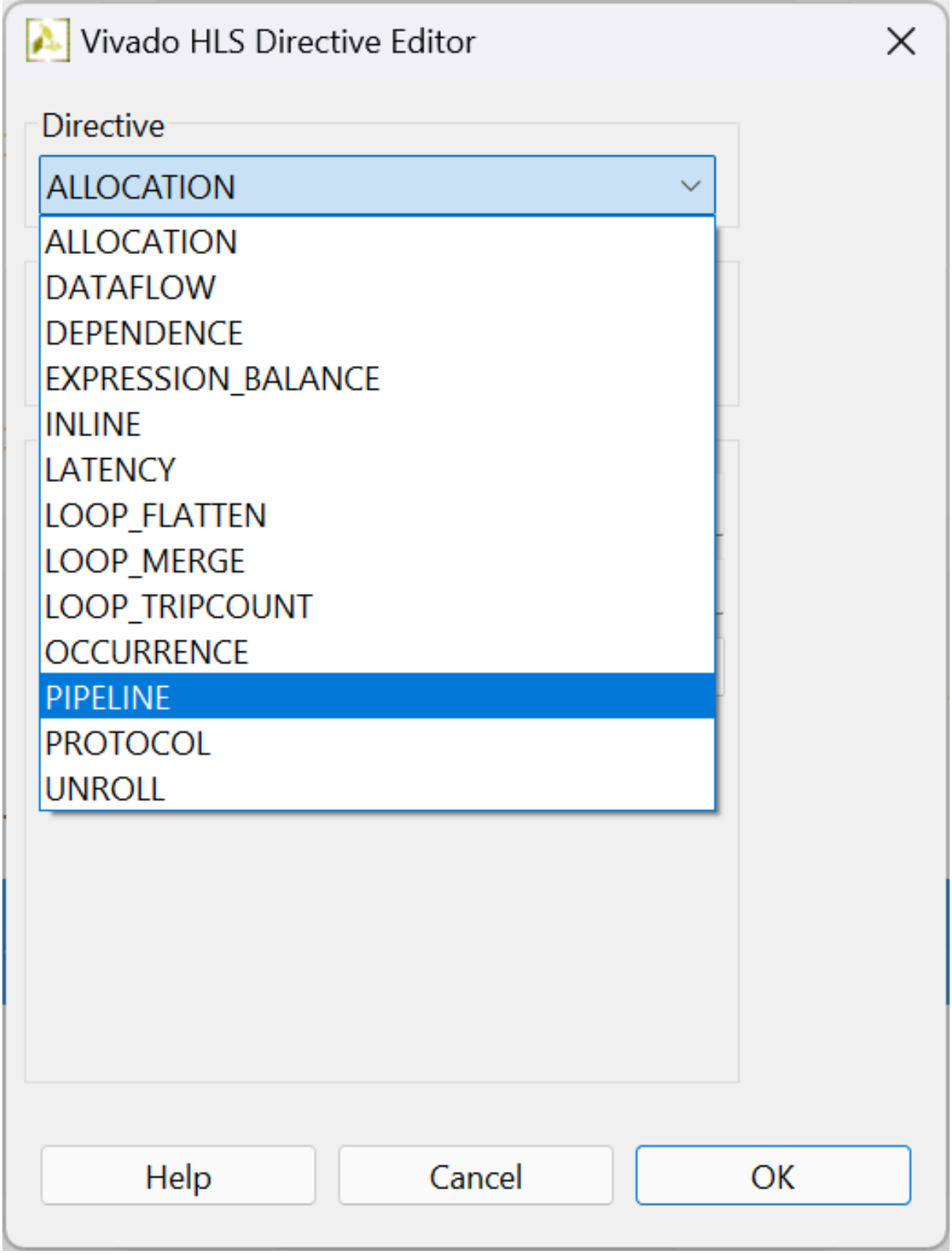
solution1

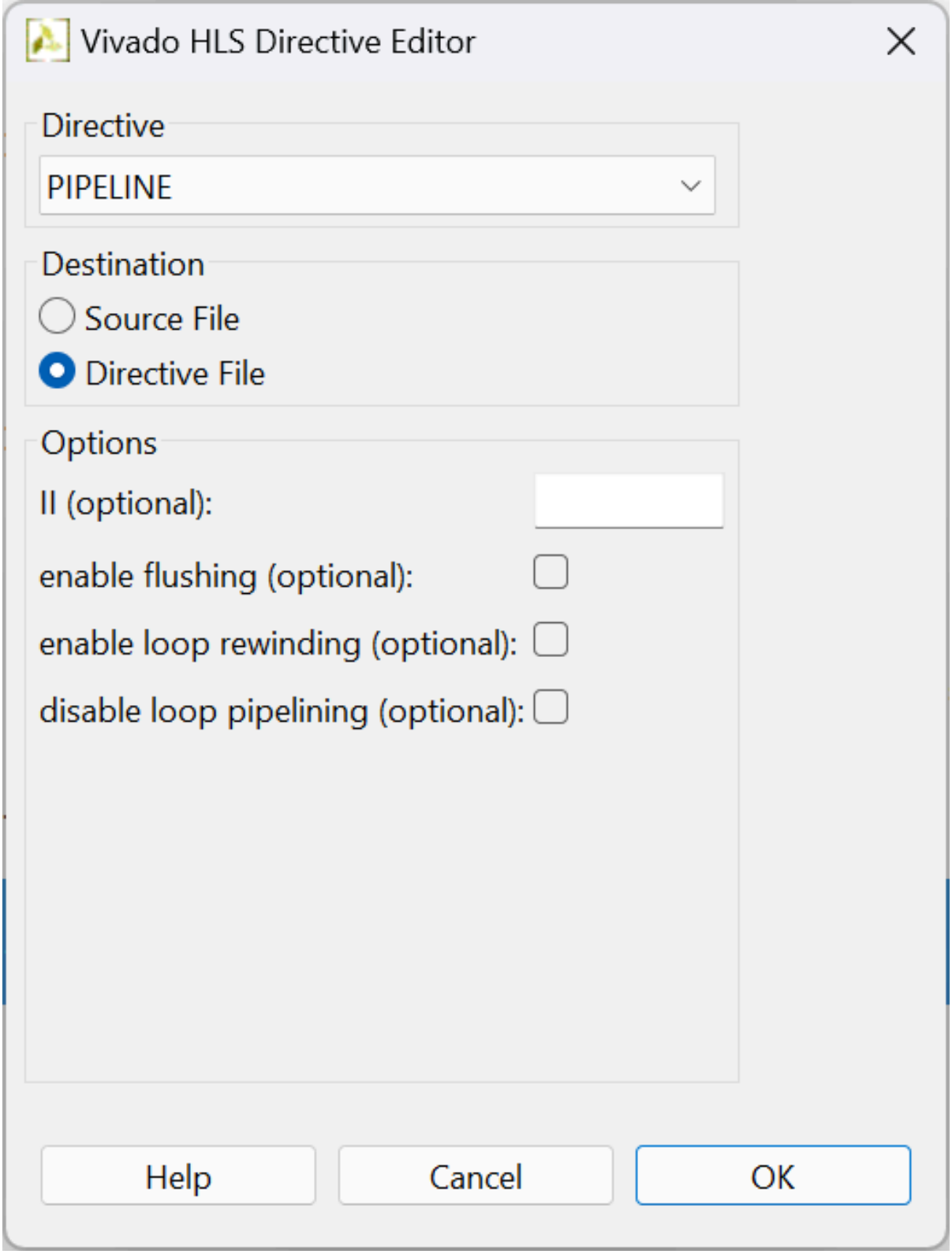
int naive size 8 with pipeline solution with all pipelines inside the most inner loop of each loops. it is necessary to insert the pragma inside the most inner loop of the loop:

```
    // files were created with all algorithms inside to avoid
    // naive matrix multiplication algorithm
    loop_mult_row: for(row = 0; row < SIZE; row++){
        loop_mult_col: for(col = 0; col < SIZE; col++){
            int sum = 0;    // same type as matrix
            loop_mult_k: for(k = 0; k < SIZE; k++){
                #pragma HLS PIPELINE    // optimization
                sum += A[row][k] * B[k][col];
            }
            C[row][col] = sum;
        }
    }
```

alternatively it is possible to insert it with the directive part


Vivado HLS 2019.1 - matrix_multiplication (C:\Users\guitr\Documents\vivado\matrix_multiplication)
File Edit Project Solution Window Help
Debug Synthesis Analysis
Explorer
matrix_multiplication
Includes
Source
matrix_mult.cpp
matrix_mult.h
Test Bench
matrix_mult_tb.c
matrix_mult_tb.h
solution0
solution1
constraints
directives.tcl
script.tcl
csim
build
report
impl
misc
verilog
vhdl
sim
autowrap
report
tv
vhdl
wrapc
wrapc_pc
syn
matrix_mult.h
matrix_mult.cpp
matrix_mult_tb.h
matrix_mult_tb.cpp
Synthesis(solution1)(matrix_mult_1_csynth.rpt)
// only one stream used, if two streams are used code could be modified
// this is different usage of hardware vs software execution time
// stream data read
// matrix A
loop_stream_read_row_A: for(row = 0; row < SIZE; row++){
loop_stream_read_col_A: for(col = 0; col < SIZE; col++){
#pragma HLS PIPELINE // optimization
A[row][col] = input_stream.read().data;
}
}
// matrix B
loop_stream_read_row_B: for(row = 0; row < SIZE; row++){
loop_stream_read_col_B: for(col = 0; col < SIZE; col++){
#pragma HLS PIPELINE // optimization
B[row][col] = input_stream.read().data;
}
}
// files were created with all algorithms inside to avoid function calls and reduce memory access therefore improving exe
// naive matrix multiplication algorithm
loop_mult_row: for(row = 0; row < SIZE; row++){
loop_mult_col: for(col = 0; col < SIZE; col++){
int sum = 0; // same type as matrix
loop_mult_k: for(k = 0; k < SIZE; k++){
#pragma HLS PIPELINE // optimization
sum += A[row][k] * B[k][col];
}
C[row][col] = sum;
}
}
//stream data write
loop_stream_write_row: for(row = 0; row < SIZE; row++){
loop_stream_write_col: for(col = 0; col < SIZE; col++){
#pragma HLS PIPELINE // optimization
data_axis.data = C[row][col];
if((row == SIZE-1) && (col == SIZE-1))
data_axis.last = 1;
else
data_axis.last = 0;
output_stream.write(data_axis);
}
}
Outline
Directive
matrix_mult_1
HLS INTERFACE ap_ctrl_none port=return
input_stream
HLS INTERFACE axis register both port=input_stream
output_stream
HLS INTERFACE axis register both port=output_stream
A
B
C
data_axis
data
last
loop_stream_read_row_A
loop_stream_read_col_A
HLS PIPELINE
loop_stream_read_row_B
loop_stream_read_col_B
HLS PIPELINE
loop_mult_row
loop_mult_col
loop_mult_k
HLS P
Insert Directive...
loop_stream_write_row
loop_stream_write_col
HLS PIPELINE
Console
Errors
Warnings
DRCs
1 DRC-Infos
0 DRC-Warnings
0 DRC-Errors
solution1


Vivado HLS Directive Editor
Directive
ALLOCATION
ALLOCATION
DATAFLOW
DEPENDENCE
EXPRESSION_BALANCE
INLINE
LATENCY
LOOP_FLATTEN
LOOP_MERGE
LOOP_TRIPCOUNT
OCCURRENCE
PIPELINE
PROTOCOL
UNROLL
Help
Cancel
OK



whit will automatically generate the data pragma flag to insert in the code.

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	8.742	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
193	193	128	128	function

Detail

Instance

Loop

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	864	0	21488	-
FIFO	-	-	-	-	-
Instance	-	-	-	-	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	994	-
Register	-	-	24913	-	-
Total	0	864	24913	22482	0
Available	280	220	106400	53200	0
Utilization (%)	0	392	23	42	0

Detail

int naive size 8 with pipeline solution with all pipelines outside the most otter loop of each loops. in this case it use more ressources than those available in this device.

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	8.510	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
271	271	271	271	none

Detail

Instance

Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- loop_stream_read_row_A_loop_stream_read_col_A	64	64	1	1	1	64	yes
- loop_stream_read_row_B_loop_stream_read_col_B	64	64	1	1	1	64	yes
- loop_mult_row	38	38	11	4	1	8	yes
- loop_stream_write_row_loop_stream_write_col	65	65	3	1	1	64	yes

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	192	0	3656	-
FIFO	-	-	-	-	-
Instance	-	-	-	-	-
Memory	6	-	0	0	0
Multiplexer	-	-	-	931	-
Register	0	-	5571	32	-
Total	6	192	5571	4619	0
Available	280	220	106400	53200	0
Utilization (%)	2	87	5	8	0

solution2

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	8.510	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
239	239	239	239	none

Detail

Instance

Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- loop_stream_read_row_A_loop_stream_read_col_A	64	64	1	1	1	64	yes
- loop_stream_read_row_B_loop_stream_read_col_B	64	64	1	1	1	64	yes
- loop_mult_row	34	34	7	4	1	8	yes
- loop_stream_write_row_loop_stream_write_col	65	65	3	1	1	64	yes

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	192	0	3626	-
FIFO	-	-	-	-	-
Instance	-	-	0	360	-
Memory	2	-	1024	64	0
Multiplexer	-	-	-	1399	-
Register	-	-	7399	-	-
Total	2	192	8423	5449	0
Available	280	220	106400	53200	0
Utilization (%)	~0	87	7	10	0

Vivado HLS 2019.1 - TCL